Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void beginning() {
    struct node * first,*second,*third;

    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));

    first->data = 40;
    second->data = 60;
    third->data = 80;

    head =  first;

     first->next = second;
    second->prev =  first;

```

Output

Clear

```
List: 10 40 60 80

=== Code Execution Successful ===
```

Programi
Premiu
Course
Progra
Learn Mo

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again. Learn with interactive visuals
Try Now
Programiz PRO ›

main.c

Share Run

```c
    second->next =third;
third->prev = second;

    third->next = head;
    head->prev = third;
}

void insertbegin(int data){
    struct node *newnode,*third;

    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;

   third = head->prev;

    newnode->next = head;
    newnode->prev =third;

    third->next = newnode;
    head->prev = newnode;

    head = newnode;
}

void display(){
    struct node *temp = head;

    if(head == NULL){
```

Output

Clear

```
List: 10 40 60 80

=== Code Execution Successful ===
```

Programiz PRO
Premium Courses by Programiz
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again. Learn with interactive visuals
Try Now
Programiz PR

main.c

Share Run

```c

void display(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty");
        return;
    }

    printf("%d ", temp->data);
    temp = temp->next;

    while(temp != head){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    beginning();
    insertbegin(10);

    printf("List: ");
    display();

    return 0;
}
```

Output

Clear

```
List: 10 40 60 80

=== Code Execution Successful ===
```

Progran
Premiu
Course
Progra
Learn M

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO ›

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void end(){
    struct node *first,*second;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 20;
    first->data = 45;
    second->data = 80;

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = second;
```

Output

Clear

```
Before deletion: 20 45 80
After deletion: 20 45

=== Code Execution Successful ===
```

Programiz PRO
Premium Courses by Programiz
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO ›

main.c

Share Run

```c

    second->prev = first;
    second->next = NULL;
}

void delete_end(){
    struct node *temp = head;

    if(head == NULL){
        printf("List is empty\n");
        return;
    }

    while(temp->next != NULL){
        temp = temp->next;
    }

    temp->prev->next = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
```

Output

Clear

```
Before deletion: 20 45 80
After deletion: 20 45

=== Code Execution Successful ===
```

Programiz PRO
Premium Courses by Programiz
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO ›

main.c

Share Run

```c

    temp->prev->next = NULL;
    free(temp);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){
    end();

    printf("Before deletion: ");
    display();

    delete_end();

    printf("\nAfter deletion: ");
    display();

    return 0;
}
```

Output

Clear

```
Before deletion: 20 45 80
After deletion: 20 45

=== Code Execution Successful ===
```

Programiz PRO
Premium Courses by Programiz
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again. Learn with interactive visuals
Try Now

Programiz

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>

struct node{
    int data;
    struct node *prev;
    struct node *next;
};

struct node *head = NULL;

void create(){
    struct node *first, *second;

    head = (struct node*)malloc(sizeof(struct node));
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 21;
    first->data = 57;
    second->data = 45;

    head->prev = NULL;
    head->next = first;

    first->prev = head;
    first->next = second;
```

Output

Clear

```
Before deletion: 21 57 45
After deletion: 21 45

=== Code Execution Successful ===
```

Programiz PRO
Learn Data Structures and Algorithms with live code visualizations
Try Now
Programiz PRO ›

main.c

Share Run

```c

    second->prev = first;
    second->next = NULL;
}

void delete_pos(int pos){
    struct node *temp = head;
    int i;

    if(head == NULL){
        printf("List is empty\n");
        return;
    }

    if(pos == 1){
        head = head->next;
        if(head != NULL)
            head->prev = NULL;
        free(temp);
        return;
    }

    for(i = 1; i < pos-1; i++){
        temp = temp->next;
    }

    struct node *del = temp->next;

```

Output

Clear

```
Before deletion: 21 57 45
After deletion: 21 45

=== Code Execution Successful ===
```

Programiz PRO
Premium Courses by Programiz
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again.
Learn with interactive visuals
Try Now
Programiz PRO

main.c

Share Run

```c

    struct node *del = temp->next;

    temp->next = del->next;

    if(del->next != NULL)
        del->next->prev = temp;

    free(del);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){

    create();

    printf("Before deletion: ");
    display();

```

Output

Clear

```
Before deletion: 21 57 45
After deletion: 21 45

=== Code Execution Successful ===
```

Programi
Premium
Courses
Program
Learn More

Programiz
C Online Compiler

Programiz PRO
Never struggle with DSA again. Learn with interactive visuals
Try Now

main.c

Share Run

```c
    free(del);
}

void display(){
    struct node *temp = head;

    while(temp != NULL){
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main(){

    create();

    printf("Before deletion: ");
    display();

    delete_pos(2);

    printf("\nAfter deletion: ");
    display();

    return 0;
}
```

Output

```
Before deletion: 21 57 45
After deletion: 21 45

=== Code Execution Successful ===
```